Neural Surrogate Framework for Polymer Dynamics

Abhishek Sharma
SURGE Programme, IIT Kanpur
Mentor: Prof. Indranil Saha Dalal Department of Chemical Engineering, IIT Kanpur

July 2026

Abstract

Traditional Brownian Dynamics (BD) simulations of polymer models require extremely small time steps ($dt = 0.001\,\mathrm{s}$) to maintain numerical stability, making long simulations computationally expensive. This work presents a **Neural Surrogate Framework for Polymer Dynamics**, where a neural network acts as a fast-forward propagator by directly predicting polymer configurations five time steps ahead ($\Delta t_{\mathrm{macro}} = 0.005\,\mathrm{s}$).

To train the model while preserving thermal equilibrium, an "Option B" strategy is employed. Random Brownian noise is removed during dataset generation so that the network learns only the deterministic spring mechanics. During deployment, the analytical thermal noise is reintroduced explicitly. The resulting hybrid framework significantly accelerates simulation while maintaining physical accuracy, achieving less than 5% deviation from the theoretical equilibrium baseline.

1 Physical Model

1.1 Bead–Spring Polymer Representation

The polymer chain consists of six beads connected by five nonlinear springs. The position of bead i is defined as $\mathbf{r}_i = (x_i, y_i, z_i)$ for $i = 1, \ldots, 6$. Connector vectors and their squared lengths are defined as:

$$\mathbf{R}_k = \mathbf{r}_{k+1} - \mathbf{r}_k, \qquad r_k^2 = |\mathbf{R}_k|^2. \tag{1}$$

1.2 Spring Force Model

A nonlinear elastic force law is used to govern the connection:

$$C(r_k^2) = \frac{3v^2 - r_k^2}{v(v^2 - r_k^2)}, \qquad \mathbf{F}_k = C(r_k^2)\mathbf{R}_k. \tag{2}$$

This force behaves similarly to a Finite Extensibility Nonlinear Elastic (FENE) spring and prevents unphysical extension beyond the maximum extensibility parameter v, where $v = 100$.

1.3 Force Balance on Beads

Each bead experiences forces from adjacent springs. For example, interior bead 2 experiences the following balance:

$$\mathbf{F}_2 = -C_1\mathbf{R}_1 + C_2\mathbf{R}_2. \tag{3}$$

Interior beads experience forces from both neighboring springs, while terminal beads experience forces from only one spring.

1.4 Brownian Dynamics Equation

Polymer motion follows the overdamped Langevin equation $d\mathbf{r}_i = \mathbf{F}_i\,dt + \sqrt{2D}\,d\mathbf{W}_i$, where D is the diffusion coefficient and $d\mathbf{W}$ denotes a Wiener process. The discrete numerical update used in standard simulation loops is:

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \mathbf{F}_i\Delta t + \sqrt{6\Delta t}\,\boldsymbol{\xi}, \tag{4}$$

where $\boldsymbol{\xi}$ is a random vector uniformly distributed in the interval $[-1, 1]$.

1.5 Quantity of Interest

The physical size of the polymer chain is monitored through its end-to-end distance $R_{\text{end}} = |\mathbf{r}_6 - \mathbf{r}_1|$. The principal statistical observable is the Root-Mean-Square (RMS) end-to-end distance:

$$R_{\text{RMS}} = \sqrt{\langle R_{\text{end}}^2 \rangle}. \tag{5}$$

2 Motivation and Objective

2.1 Limitations of Adaptive Time Stepping

Earlier work investigated an AI-based Adaptive Time Stepping (ATS) controller. The ATS network dynamically adjusted the simulation time step based on instantaneous force magnitudes. Although ATS improved performance, the simulation framework remained inherently sequential:

1. Compute physical spring forces.
2. Query the neural network controller.
3. Advance one varying step forward.
4. Repeat.

Because it still required step-by-step force evaluations at every single iteration, the overall efficiency gain remained limited.

2.2 Project Objective

The primary objective of this study is to replace the sequential Brownian Dynamics solver entirely with an autonomous **Surrogate Neural Network Propagator**. Instead of computing five individual micro-steps, the network directly predicts the polymer configuration at a larger macro-step $t + 5dt$. To ensure perfect translation invariance, the model operates exclusively on internal connector vectors $\mathbf{Q}_i = \mathbf{r}_{i+1} - \mathbf{r}_i$, allowing the network to learn only the structural changes of the polymer without being affected by its absolute location in the simulation box.

3 Methodology

The complete framework consists of five operational stages running continuously:

1. Gather Data (Noise ON) → 2. 5-Step Trajectories (Noise OFF) → 3. Preprocessing → 4. Train NN → 5. Hybrid Deployment

3.1 Step 1: Generating the Equilibrium Pool

A long, high-fidelity Brownian Dynamics simulation was run with full thermal noise active. After allowing the chain to relax and equilibrate for 200 s, configuration snapshots were saved every 100 s to obtain a dataset of $15,000$ independent equilibrium configurations.

3.2 Step 2: Trajectory Generation (Option B)

Training a neural network directly on stochastic trajectories causes an artificial variance-collapse problem. Since thermal noise is completely random, minimizing Mean Squared Error (MSE) loss forces the network to calculate the mathematical average of the noise, which is zero. Consequently, the model loses thermal energy during long deployment runs and eventually freezes into an unphysical absolute zero-temperature state.

To avoid this trap, we use **Option B**:

1. Load an equilibrium configuration from the saved pool.
2. Turn the random Brownian noise completely OFF.
3. Simulate forward for exactly five micro-steps ($5 \cdot dt$) under spring forces alone.
4. Store the initial state as the network input X.
5. Store the final state as the network target y.

This cleanly isolates the complex deterministic spring mechanics for the model to learn.

3.3 Step 3: Vectorization and Reshaping

Absolute coordinates are converted into 3D connector vectors $\mathbf{Q}_k = \mathbf{r}_{k+1} - \mathbf{r}_k$. For a chain of five springs across three axes, this yields $5 \times 3 = 15$ dimensions. The dataset is split using an 80:20 ratio into a **Training Set** of 12,000 samples and a **Validation Set** of 3,000 samples.

3.4 Step 4: Neural Network Architecture

The surrogate model is a Multi-Layer Perceptron (MLP) mapping 15 inputs to 15 outputs.

Layer	Neuron Count	Activation Function
Input	15	–
Hidden 1	32	SiLU
Hidden 2	32	SiLU
Output	15	Tanh

Table 1: Neural Network Architecture

The trainable parameters for each layer are calculated via Parameters = (Inputs × Outputs) + Biases:

- **Layer 1:** $(15\times32)+32 = 512$ **Layer 2:** $(32\times32)+32 = 1056$ **Layer 3:** $(32\times15)+15 = 495$

$$\text{Total Trainable Parameters} = 512 + 1056 + 495 = \mathbf{2063} \tag{6}$$

The network is trained for 150 epochs using the AdamW optimizer with a learning rate of 10^{-3} and a weight decay regularizer of 10^{-6}.

3.5 Step 5: Hybrid Deployment Loop

The trained surrogate network completely replaces explicit force loops at runtime. At each macro-step ($\Delta t_{\text{macro}} = 5 \times dt = 0.005\,\text{s}$):

1. The current connector vectors are fed into the network to obtain the deterministic structural shape prediction.
2. Thermal noise is reintroduced analytically to restore the missing energy profile:

$$\sigma_{\text{connector}} = \sqrt{4\Delta t_{\text{macro}}} \tag{7}$$

3. Absolute bead coordinates are reconstructed using the cumulative vector sums starting from Bead 0.

4 Results and Discussion

To validate the framework, long simulations of $10,000$ s were conducted. For a polymer chain with 5 independent connectors where the maximum extensibility (Kuhn length) parameter is $v = 100$, the theoretical equilibrium prediction for the expected mean-squared distance is $\langle R_{ee}^2 \rangle = 5 \times v = 5 \times 100 = 500$. Therefore, the exact target baseline for the RMS end-to-end distance is $R_{\text{RMS}} = \sqrt{500} \approx 22.36$.

The hybrid surrogate simulation produced an average $R_{\mathbf{RMS}} \approx 21$. This matches the true physical theory within a tight 5% **error margin**. Furthermore, the standard deviation profile remained completely stable throughout the $10,000$ s timeline, showing that our decoupled noise reinjection approach successfully eliminated the variance-collapse trap.

5 Conclusions

The Neural Surrogate Framework demonstrated that:

1. Significant computational speedups are achieved by replacing step-by-step force calculations with single forward-pass neural propagations.
2. Separating deterministic dynamics from stochastic noise during training completely eliminates the unphysical freezing and chain-shrinkage problem.

3. Removing mismatched preprocessing scales restores physically accurate equilibrium statistics.
4. Long-term trajectories remain thermodynamically stable and reproduce the theoretical RMS end-to-end distance profile (~ 22.36) within an error of 5%.

6 Future Scope

Future work over the next 2 to 3 months will focus on preparing this framework for publication by expanding on the following fields:

1. **Step Limit Investigation:** Determine the maximum stable macro-step multiplier limit ($10dt$, $50dt$, etc.) before the network's structural accuracy begins to drop.
2. **Complex Polymer Topologies:** Extend the learned propagator methodology to non-linear structures, including ring polymers, branched polymers, and star polymers.
3. **Graph Neural Networks (GNNs):** Replace the flat MLP model with a GNN architecture to naturally handle variable chain lengths and external fluid flow environments without retraining core weights.